

LeetCode Deep Dive

From First Instinct to Optimal Algorithm: Dynamic Programming in Practice
Week 8 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

Why walk through LeetCode problems?

- This week's LEETCODE.md list gives ten Week 8 problems to practice on your own. This deck narrates the thinking process for two of them, end to end, so you can reuse it on the rest.
- We pick one **Medium** (*Coin Change* – unbounded knapsack) and one **Hard** (*Maximum Profit in Job Scheduling* – weighted interval scheduling, this week's *other* lecture topic, revisited).
- Both times the naive instinct is the same one the lecture already warned about: a recursive solution with **overlapping subproblems** that recomputes the same answer over and over – exactly the Fibonacci blow-up from this week's notes.

The four-step process we will repeat twice

- ❶ **First instinct.** Write the recursive recurrence that follows directly from the problem statement, with no memoization.
- ❷ **Measure why it hurts.** Count how many times the same subproblem gets recomputed, and see the exponential blow-up.
- ❸ **Find the structural insight.** Identify the *subproblem* (this week's step 1 of the 5-step DP recipe) whose optimal-substructure recurrence (step 2) lets one pass fill every value exactly once.
- ❹ **Implement the optimal algorithm,** trace the DP table by hand on a small example, then look for the programming-level tricks that make the implementation itself correct and fast.

This mirrors the course

Steps 1–3 are exactly this week's “define the subproblem, write the recurrence, analyze the cost” recipe; step 4 is what turns a correct recurrence into working code.

LeetCode 322 – Coin Change (Medium)

You are given coin denominations `coins` (an unlimited supply of each) and a target `amount`. Return the **fewest** coins needed to make up `amount`, or `-1` if it cannot be made up at all.

- Constraints (LeetCode): $1 \leq \text{coins.length} \leq 12$, $1 \leq \text{amount} \leq 10^4$.
- This is exactly the **unbounded knapsack** variant this week's lecture covers under "Coin change: minimum number of coins."

Worked example

`coins = [1, 2, 5], amount = 11`

Answer: 3 – e.g. $5 + 5 + 1 = 11$ using three coins. (Also $5 + 2 + 2 + 2 = 11$ uses four – more, so it's not optimal.)

- Greedy “always take the largest coin that fits” happens to work for US coinage but is not correct in general (e.g. `coins=[1,3,4]`, `amount=6`: greedy gives $4 + 1 + 1 = 3$ coins, but $3 + 3 = 2$ coins is better). We need an algorithm that is *provably* optimal for *any* coin set.

First instinct: recurse on “which coin last?”

Most direct reading of the problem

$\text{MINCOINS}(a) = 1 + \min_{c \in \text{coins}, c \leq a} \text{MINCOINS}(a - c)$, with $\text{MINCOINS}(0) = 0$.

```
1: function MINCOINS(a)
2:   if a = 0 then return 0
3:   end if
4:   if a < 0 then return ∞
5:   end if
6:   return 1 + minc ∈ coins MINCOINS(a − c)
7: end function
```

This is a direct transcription of the problem – and it is **correct**.

Why this is bad

- `MinCoins(11)` calls `MinCoins(10)`, `MinCoins(9)`, `MinCoins(6)`; `MinCoins(10)` *also* calls `MinCoins(9)` and `MinCoins(8)` and `MinCoins(5)` – the same smaller amounts get recomputed from many different paths.
- This is **exactly** the overlapping-subproblems blow-up this week's notes illustrate with Fibonacci ($F(n)$ makes $2F(n+1) - 1$ calls without memoization). Here the branching factor is $|\text{coins}|$ and the depth is up to `amount`, so the call count grows like $|\text{coins}|^{\text{amount}}$ in the worst case.
- At `amount = 104` this is not a slowdown – it is astronomically infeasible.

The smell to notice

Whenever a recursion's arguments repeat across different call paths, that is the signature of overlapping subproblems – memoize by that argument.

The structural insight

- There are only $\text{amount}+1$ *distinct* subproblems ($a = 0, 1, \dots, \text{amount}$), each doing $O(|\text{coins}|)$ work once its smaller answers are known.
- **Subproblem (step 1 of this week's recipe):** $dp[a]$ = fewest coins to make amount a .
- **Recurrence (step 2):** $dp[a] = 1 + \min_{c \leq a} dp[a - c]$, $dp[0] = 0$, evaluated bottom-up for $a = 1, \dots, \text{amount}$ so every smaller value is already known – no recomputation, ever.

Trace on the worked example

coins = [1, 2, 5] – $dp[a]$ for $a = 0, \dots, 11$ (bottom-up):

a	0	1	2	3	4	5	6	7	8	9	10	11
$dp[a]$	0	1	1	2	2	1	2	2	3	3	2	3

- $dp[5] = 1$ (one 5-coin) is what makes $dp[10] = 2$ (5 + 5) and, in turn, $dp[11] = 1 + dp[10] = 1 + 2 = 3$ (5 + 5 + 1) beat $1 + dp[9] = 4$ and $1 + dp[6] = 3$ (tied, both valid optimal decompositions).
- Result: $dp[11] = 3$, matching the expected answer.

Complexity: naive vs. optimal

Approach	Time	At amount= 10^4
Unmemoized recursion	$O(\text{coins} ^{\text{amount}})$	astronomically infeasible
Bottom-up DP	$O(\text{amount} \times \text{coins})$	$\leq 1.2 \times 10^5$ ops

Exactly the same shape of improvement as this week's Knapsack DP: exponential recursion collapses to (number of subproblems) $\times$ (work per subproblem) once each subproblem is solved only once.

Python implementation

```
1 def coinChange(coins, amount):
2     INF = amount + 1                # sentinel: "not (yet) reachable"
3     dp = [0] + [INF] * amount       # dp[a] = fewest coins for amount a
4
5     for a in range(1, amount + 1):
6         for c in coins:
7             if c <= a:
8                 dp[a] = min(dp[a], dp[a - c] + 1)
9
10    return dp[amount] if dp[amount] != INF else -1
```

Implementation tips & tricks (the programming side)

- **Sentinel choice:** `amount + 1`, **not** `float('inf')`. `amount+1` is already larger than any valid answer (which is at most `amount`, using all 1-coins), and unlike `float('inf')` it stays an `int`, so `dp[a-c] + 1` never silently becomes a float.
- **Iterative DP, not memoized recursion, for the same reason as Week 3's Union-Find.** A recursive `@lru_cache` version recurses to depth `amount` in the worst case (e.g. `coins=[1]`); at `amount = 104` that exceeds Python's default recursion limit (≈ 1000) and crashes. The bottom-up loop has no such limit.
- **Loop order doesn't matter here, but it would for "Coin Change II."** Because each coin can be reused arbitrarily (unbounded knapsack), iterating `amount` outside and `coins` inside is safe. In the closely related *counting* problem (number of *combinations*, not permutations), swapping the loop order changes the answer – a classic, easy-to-miss bug.

LeetCode 1235 – Maximum Profit in Job Scheduling (Hard)

You are given n jobs, each with a `startTime`, `endTime`, and `profit`. You may schedule any subset of **non-overlapping** jobs (job A and job B don't overlap if A 's end $\leq B$'s start, or vice versa). Return the **maximum total profit**.

- Constraints (LeetCode): $1 \leq n \leq 5 \times 10^4$.
- This is **exactly** this week's other lecture topic – Weighted Interval Scheduling – with profit playing the role of w_j .

Worked example

`startTime=[1,2,3,3], endTime=[3,4,5,6], profit=[50,10,40,70]`

Answer: 120 – take job 1 ($[1, 3)$, profit 50) and job 4 ($[3, 6)$, profit 70): they don't overlap ($3 \leq 3$), total $50 + 70 = 120$. Any other combination scores less (e.g. job 3 alone plus job 4 overlaps, since $3 < 5$ is not ≤ 3).

First instinct, and why the naive version is bad

Most direct reading of the problem

Try every subset of jobs, keep the non-overlapping ones with maximum profit sum.

- That is $O(2^n)$ – utterly infeasible at $n = 5 \times 10^4$.
- Even the “obviously smarter” fix – sort by end time, then for each job scan *all previous* jobs to find the latest one that ends before it starts – is a real DP (correct!), but costs $O(n)$ per job for that scan, so $O(n^2)$ total. At $n = 5 \times 10^4$ that's 2.5×10^9 operations: still too slow.

The smell to notice

A DP whose recurrence is right but whose *per-subproblem* work is a linear scan is a sign that the scan itself can probably be replaced by a binary search – exactly this week's lecture on Weighted Interval Scheduling.

The structural insight

- Sort jobs by **end time**. Define $p(j)$ = the largest index $i < j$ with $\text{end}_i \leq \text{start}_j$ (the latest job compatible with j).
- **Subproblem:** $M[j] = \text{max profit using only jobs } 1, \dots, j$.
- **Recurrence:** $M[j] = \max(M[j-1], \text{profit}_j + M[p(j)])$ – either skip job j , or take it and add the best profit compatible with it.
- Because jobs are sorted by end time, $p(j)$ is exactly “the rightmost already-processed end time that is $\leq \text{start}_j$ ” – a **binary search** into the (already sorted) array of end times, in $O(\log n)$.

Trace on the worked example

Sorted by end time: job1(1, 3, 50), job2(2, 4, 10), job3(3, 5, 40), job4(3, 6, 70).

j	start_j	profit_j	$p(j)$	$M[j]$	reasoning
1	1	50	0	50	$\max(M[0]=0, 50 + M[0]=50)$
2	2	10	0	50	$\max(M[1]=50, 10 + M[0]=10)$
3	3	40	1	90	$\max(M[2]=50, 40 + M[1]=90)$
4	3	70	1	120	$\max(M[3]=90, 70 + M[1]=120)$

$p(3) = p(4) = 1$ because job1 is the latest job ending at or before start 3. Result: $M[4] = 120$, matching the expected answer.

Complexity: naive vs. optimal

Approach	Time	At $n = 5 \times 10^4$
Try all subsets	$O(2^n)$	infeasible
DP with linear scan for $p(j)$	$O(n^2)$	$\sim 2.5 \times 10^9$ ops
DP with binary search for $p(j)$	$O(n \log n)$	$\sim 8 \times 10^5$ ops

Same $O(n \log n)$ bound as this week's lecture proves for Weighted Interval Scheduling in general.

Python implementation

```
1  from bisect import bisect_right
2
3  def jobScheduling(startTime, endTime, profit):
4      jobs = sorted(zip(endTime, startTime, profit))    # sort by end time
5      ends = [e for e, s, p in jobs]                  # already sorted, reused
6                                                       below
7
8      n = len(jobs)
9      M = [0] * (n + 1)                                # M[j] = best profit using jobs[0:j]
10     for j in range(1, n + 1):
11         end_j, start_j, profit_j = jobs[j - 1]
12         i = bisect_right(ends, start_j, 0, j - 1)    # p(j): jobs[0:i] all end <=
13                                                       start_j
14         M[j] = max(M[j - 1], profit_j + M[i])
15
16     return M[n]
```

Implementation tips & tricks (the programming side)

- **Sort the three parallel arrays together.** A common bug is sorting `endTime` alone and forgetting to permute `startTime` and `profit` to match. Zipping all three into tuples before sorting makes this impossible to get wrong.
- **Reuse `bisect_right` instead of hand-rolling binary search.** `bisect_right(ends, start_j, 0, j-1)` directly returns the count of already -processed jobs whose end time is $\leq \text{start}_j$ – which is $p(j)$ in this 1-indexed M array, with zero off-by-one arithmetic to get wrong.
- **The bound $0, j-1$ matters.** Searching only the *prefix* already sorted by end time (jobs $1, \dots, j-1$) is both correct and avoids accidentally matching a later job that happens to share the same end time as job j itself.

Summary: two problems, one lesson

- 1 **Coin Change** is unbounded knapsack: the naive recursion recomputes the same amount from many paths (this week's Fibonacci warning, restated), and bottom-up DP fixes it in $O(\text{amount} \times |\text{coins}|)$.
- 2 **Maximum Profit in Job Scheduling** is Weighted Interval Scheduling with a different name: the DP recurrence is already optimal, but a linear scan for $p(j)$ silently costs $O(n^2)$ – replacing it with binary search recovers the lecture's $O(n \log n)$ bound.
- 3 Both times, the naive instinct was correct but wasteful, in the exact shape this week's 5-step DP recipe predicts: define the subproblem, write the recurrence, and only then does the running time become (subproblems) $\times$ (work per subproblem) – and that per-subproblem work is itself worth optimizing.
- 4 The implementation tricks (integer sentinels, iterative not recursive DP, bisect instead of hand-rolled search) are a second, independent layer: even an asymptotically optimal recurrence can fail in practice if the code hits language-level limits or off-by-one errors.

Keep practicing

LEETCODE.md lists eight more Week 8 problems (*House Robber*, *Partition Equal Subset Sum*, *Target Sum*, *Ones and Zeroes*, ...) – try applying the same four-step process to each of them.